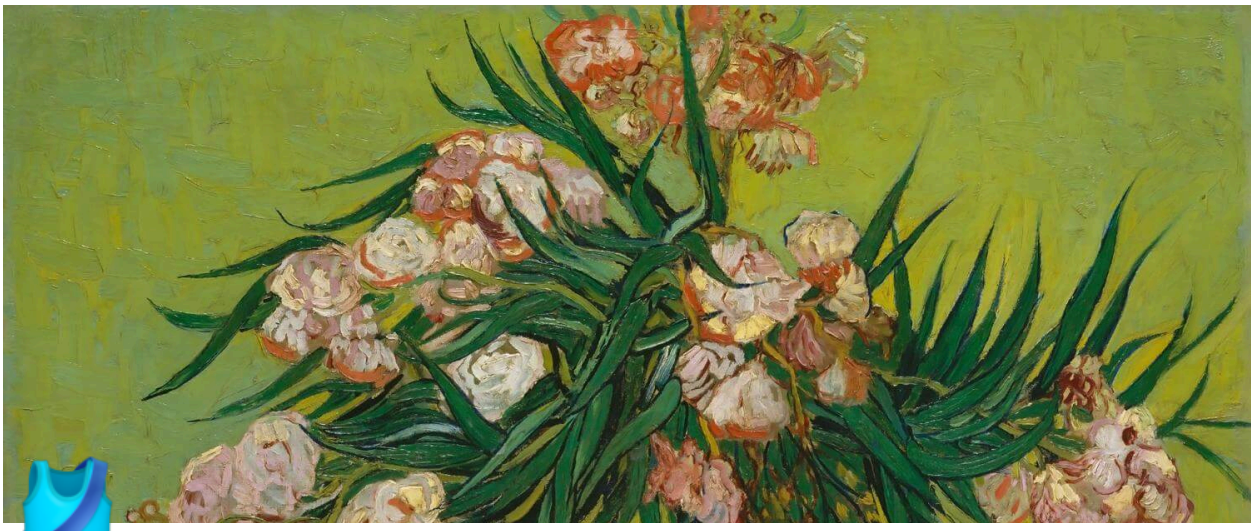


开发文档

🚀 Project Name: Web Blackjack 21 (MVP)

开发需求文档 (Product Requirements Document)

1. 项目概述 (Overview)

开发一个基于 Web 的单机版 21点（Blackjack）游戏。
核心目标： 提供沉浸式的、拟物化的赌场体验，逻辑严谨，并为未来扩展（多人、联网）预留架构接口。
当前阶段： MVP (Minimum Viable Product)，单人 vs 庄家 (1v1)。

---

2. 技术栈 (Tech Stack)

- **Core Framework:** React 18+ (使用 Vite 构建)
- **Language:** TypeScript (必须严格类型定义，方便未来维护)
- **State Management:** React Context 或 Zustand (用于管理全局游戏状态)
- **Styling:** Tailwind CSS (布局) + Custom CSS (拟物化细节)
- **Animation & Interaction: Framer Motion** (用于发牌动画、筹码拖拽、UI 进出场)

- **Assets:** 需要使用 SVG 或高质量 PNG 模拟扑克牌和筹码。

---

3. 核心游戏规则 (Game Rules - The "Standard")

必须严格分离逻辑与 UI。核心算法应封装在纯 TS 类中。

A. 基础设定 (Settings)

- **牌组 (Decks):** 6 副牌 (312张)。
- **洗牌 (Shuffle):** 模拟真切牌。当剩余牌量低于 25% (Penetration 75%) 时，当前局结束后触发洗牌。
- **庄家规则 (Dealer):**
  - **Stand on Soft 17:** 庄家软 17 点（A+6）时停牌，不继续要牌。
  - 庄家必须只要牌直到点数 >= 17。
- **赔率 (Payouts):**
  - Normal Win: 1:1
  - Blackjack: 3:2
  - Insurance: 2:1
- **特殊规则:**
  - **Split (分牌):** 任意两张点数相同的牌可分。最多分 3 次（共 4 手牌）。
  - **Split Aces:** A 分牌后，每手牌仅发 1 张牌，且不能再次分牌。
  - **Double (双倍):** 任意首两张牌可 Double。Double 后仅发 1 张牌。
  - **Surrender: 不可投降 (Disabled)**。

B. 经济系统 (Economy - Session Based)

- **初始资金:** $10,000 (可配置 `INITIAL_BALANCE`)。
- **下注限制:** Min $100, Max $2000 (可配置)。
- **生命周期:**
  - 每次加载页面视为一次“生涯”。
  - **破产条件:** 玩家余额 < 最小下注额。

- **结束画面:** 显示“生存局数 (Rounds Survived)” 和 “历史最高持有金额 (Peak Balance)”。
- **重置:** 必须手动刷新页面或点击“Restart Game”按钮（完全重载状态）。

---

4. 软件架构 (Architecture) - 关键部分

AI 必须遵循以下架构，严禁将业务逻辑写在 React 组件内部。

4.1 核心逻辑层 (`/src/engine`)

这是一个纯 TypeScript 模块，不依赖 React。

- `Card`: 定义花色、点数、数值计算。
- `Deck`: 管理 6 副牌，处理洗牌 (Shuffle)、发牌 (Deal)、剩余牌量计算。
- `Hand`: 管理一手牌。包含计算当前点数、判断是否爆牌 (Bust)、是否是 Blackjack、是否是 Soft Hand。
- `BlackjackEngine`: 主控制器。
  - State: `waiting-bet`, `dealing`, `player-turn`, `dealer-turn`, `resolving`.
  - Methods: `placeBet()`, `dealInitialCards()`, `hit()`, `stand()`, `double()`, `split()`, `resolveDealer()`.
  - **扩展性预留:** `resolvePayout()` 方法应设计为可以接受“Side Bet”参数的结构，虽然目前传空，但逻辑要预留。

4.2 状态管理层 (`/src/store`)

使用 Zustand 或 Context API 桥接 Engine 和 UI。

- 监听 Engine 的状态变化，触发 React 重渲染。
- 记录 `roundsPlayed` 和 `peakBalance`。

4.3 UI 组件层 (`/src/components`)

- **Table:** 绿色桌布背景，响应式布局。
- **ChipRack:** 放置筹码的区域，支持**拖拽 (Drag & Drop)**。
- **BettingSpot:** 下注区，作为筹码拖拽的目标 (Drop Zone)。
- **HandDisplay:** 展示玩家和庄家的牌（需处理分牌后的多手牌布局）。

- **Controls:** 动态按钮组 (Hit/Stand/Double/Split)，仅在合法状态下显示。
- **Modal:** 用于显示游戏结束统计面板。

---

5. UI/UX 详细交互规范

- **拟物化风格:**
  - 桌面材质：绿色毛毡纹理。
  - 扑克牌：清晰的大字体，带阴影，模拟真实厚度。
  - 筹码：圆形，带条纹，不同面额颜色不同（$10, $50, $100, $500）。
- **动画 (Animations):**
  - 发牌：牌从“发牌靴”位置飞入指定位置，带有轻微旋转和时差。
  - 翻牌：庄家暗牌翻开时有 3D 翻转效果。
  - 筹码：下注时筹码从筹码栏“飞”入下注区，或被拖拽进入。
  - 输赢：赢钱时筹码从庄家侧飞向玩家；输钱时筹码被收回。
- **拖拽交互:**
  - 玩家按住筹码 → 拖动到下注圈 → 松开 → 视为下注增加。
  - 同时也保留点击筹码直接下注的快捷方式（Accessibility）。
- **响应式:**
  - **Desktop:** 宽屏布局，筹码在底部或侧边。
  - **Mobile:** 竖屏/横屏自适应。如果是竖屏，需调整牌的大小以容纳分牌后的布局。

---

6. 开发步骤建议 (Implementation Steps for AI)

1. **Phase 1: Logic Core:** 编写 `Deck` 和 `Hand` 类，编写单元测试确保 A 的 1/11 点数计算正确，Split 逻辑正确。
2. **Phase 2: Basic UI:** 搭建 React 框架，实现发牌和基础按钮操作（无动画），联通逻辑。
3. **Phase 3: Interaction & Animation:** 引入 Framer Motion，实现发牌动画和筹码拖拽。

4. **Phase 4: Game Loop & Polish:** 完善洗牌逻辑、破产判定、Game Over 统计面板。